

i

```
In [4]: import numpy as np

#k=1/4*pi*E0
k = 8.9875e9
Q1 = 2e-6
Q2 = -4e-6
Q3 = 6e-6

r1_charge = np.array([1.0, 0.0])
r2_charge = np.array([0.0, 2.0])
r3_charge = np.array([-2.0, -1.0])

r_origin = np.array([0.0, 0.0])

print("\nElectric Field due to Q1")
d1 = r_origin - r1_charge
mag_d1 = np.linalg.norm(d1)
unit_d1 = d1 / mag_d1
E1 = k * Q1 / (mag_d1**2) * unit_d1
print(f"Displacement vector for Q1 to origin (d1): {d1} m")
print(f"Distance from Q1 to origin (mag_d1): {mag_d1} m")
print(f"Unit vector (hat_d1): {unit_d1}")
print(f"Electric field E1: {E1} N/C")

print("\nElectric Field due to Q2")
d2 = r_origin - r2_charge
mag_d2 = np.linalg.norm(d2)
unit_d2 = d2 / mag_d2
E2 = k * Q2 / (mag_d2**2) * unit_d2
print(f"Displacement vector for Q2 to origin (d2): {d2} m")
print(f"Distance from Q2 to origin (mag_d2): {mag_d2} m")
print(f"Unit vector (hat_d2): {unit_d2}")
print(f"Electric field E2: {E2} N/C")

print("\nElectric Field due to Q3")
d3 = r_origin - r3_charge
mag_d3 = np.linalg.norm(d3)
unit_d3 = d3 / mag_d3
E3 = k * Q3 / (mag_d3**2) * unit_d3
print(f"Displacement vector for Q3 to origin (d3): {d3} m")
print(f"Distance from Q3 to origin (mag_d3): {mag_d3} m (sqrt(5) approx {np.sqrt(5)})")
print(f"Unit vector (hat_d3): {unit_d3}")
print(f"Electric field E3: {E3} N/C")
```

Electric Field due to Q1
Displacement vector for Q1 to origin (d1): [-1. 0.] m
Distance from Q1 to origin (mag_d1): 1.0 m
Unit vector (hat_d1): [-1. 0.]
Electric field E1: [-17975. 0.] N/C

Electric Field due to Q2
Displacement vector for Q2 to origin (d2): [0. -2.] m
Distance from Q2 to origin (mag_d2): 2.0 m
Unit vector (hat_d2): [0. -1.]
Electric field E2: [-0. 8987.5] N/C

Electric Field due to Q3
Displacement vector for Q3 to origin (d3): [2. 1.] m
Distance from Q3 to origin (mag_d3): 2.23606797749979 m (sqrt(5) approx 2.2361)
Unit vector (hat_d3): [0.89442719 0.4472136]
Electric field E3: [9646.39725493 4823.19862747] N/C

ii

In [7]: `import numpy as np`

```
#k=1/4*pi*E0
k = 8.9875e9
Q1 = 2e-6
Q2 = -4e-6
Q3 = 6e-6

r1_charge = np.array([1.0, 0.0])
r2_charge = np.array([0.0, 2.0])
r3_charge = np.array([-2.0, -1.0])

r_origin = np.array([0.0, 0.0])

d1 = r_origin - r1_charge
mag_d1 = np.linalg.norm(d1)
unit_d1 = d1 / mag_d1
E1 = k * Q1 / (mag_d1**2) * unit_d1

d2 = r_origin - r2_charge
mag_d2 = np.linalg.norm(d2)
unit_d2 = d2 / mag_d2
E2 = k * Q2 / (mag_d2**2) * unit_d2

d3 = r_origin - r3_charge
mag_d3 = np.linalg.norm(d3)
unit_d3 = d3 / mag_d3
E3 = k * Q3 / (mag_d3**2) * unit_d3

E_total = E1 + E2 + E3
print(f"Total electric field E_total (vector sum E1 + E2 + E3): {E_total} N/C")
```

Total electric field E_total (vector sum E1 + E2 + E3): [-8328.60274507 13810.69862747] N/C

```

In [8]: import numpy as np

#k=1/4*pi*E0
k = 8.9875e9
Q1 = 2e-6
Q2 = -4e-6
Q3 = 6e-6

r1_charge = np.array([1.0, 0.0])
r2_charge = np.array([0.0, 2.0])
r3_charge = np.array([-2.0, -1.0])

r_origin = np.array([0.0, 0.0])

d1 = r_origin - r1_charge
mag_d1 = np.linalg.norm(d1)
unit_d1 = d1 / mag_d1
E1 = k * Q1 / (mag_d1**2) * unit_d1

d2 = r_origin - r2_charge
mag_d2 = np.linalg.norm(d2)
unit_d2 = d2 / mag_d2
E2 = k * Q2 / (mag_d2**2) * unit_d2

d3 = r_origin - r3_charge
mag_d3 = np.linalg.norm(d3)
unit_d3 = d3 / mag_d3
E3 = k * Q3 / (mag_d3**2) * unit_d3

E_total = E1 + E2 + E3

mag_E_total = np.linalg.norm(E_total)
print(f"Magnitude of total electric field: {mag_E_total} N/C")
angle_rad = np.arctan2(E_total[1], E_total[0])
angle_deg = np.degrees(angle_rad)
print(f"Direction of total electric field (angle with positive x-axis): {angle_deg}")

unit_E_total = E_total / mag_E_total
print(f"Unit vector of total electric field: {unit_E_total}")

```

```

Magnitude of total electric field: 16127.647697784061 N/C
Direction of total electric field (angle with positive x-axis): 121.09 degrees
Unit vector of total electric field: [-0.5164177  0.85633683]

```

```

In [ ]:

```